**NATIONAL UNIVERSITY OF SINGAPORE**
**SCHOOL OF COMPUTING**

MID-TERM QUIZ FOR
Semester 2, AY2014/2015

CS3230 – DESIGN AND ANALYSIS OF ALGORITHMS

7 March 2015 Time Allowed: 1.5 hour

---

## Instructions to Candidates:

1. This examination paper contains **FOUR** questions and comprises **12** (**TWELVE)** printed pages, including this page.
2. Answer **ALL** questions.
3. Write ALL your answers in this examination book. (Use the blank pages provided if you need to.)
4. This is an **OPEN BOOK** examination.

**Name:** _____

| QUESTION | POSSIBLE | SCORE |
|---|---|---|
| Q1 | 15 | |
| Q2 | 15 | |
| Q3 | 10 | |
| Q4 | 10 | |
| **TOTAL** | **50** | |

**Fun Bonus Question: (1 point)**

Give the full name of NP introduced in CS3230.

A. ___Non-deterministic polynomial time problem_____

**IMPORTANT NOTE:**

- *You can **freely quote** standard algorithms and data structures covered in the lectures, textbook, and homeworks. Explain **any modifications** you make to them;*
- *Unless otherwise specified, you are expected to **prove (justify)** your results.*
- *lg n is logarithm of n base 2 while ln n is logarithm of n base e (i.e. natural logarithm).*

## Q1. (15 points)

**(a)** **(8 Points)** Rank the following functions in *ascending order of growth*; that is, if function $g(n)$ *immediately* follows function $f(n)$ in your list, then it should be the case that $f(n)$ is $O(g(n))$.

$$g_1(n) = 2\sum_{k=1}^{n}(k \lg\lg k) \qquad g_2(n) = 100n(\lg n)$$

$$g_3(n) = \lg\left(\prod_{k=1}^{n}(k^2 \lg n)\right) \qquad g_4(n) = 10^{(3\ln n)}$$

To simplify notations, you should write $f(n) << g(n)$ to mean $f(n) = o(g(n))$, and $f(n) \equiv g(n)$ to mean $f(n) = \Theta(g(n))$.

*No need to prove the results.*

**SOLUTION: (in increasing order of growth rate)**

$\mathbf{g_2 \equiv g_3 << g_1 << g_4}$

## Q1. (continued)

**(b) (4 points) Asymptotic Notations (by definition)**

Let $f(n) = 2n^3 + 100\sum_{k=1}^{n} k$

By using the definitions of Θ, prove that $f(n) = \Theta(n^3)$

*Note*: You cannot *just give* the constant $c_1$ $c_2$, and $n_0$. You need to *prove* it.

Soln: $f(n)=2n^3 + 100n(n+1)/2=2n^3+50n^2+50n$.

We set c=102 and $n_0$=1.
Then, $f(n)=2n^3+50n^2+50n\leq 102n^3=cn^3$ for $n\geq 1=n_0$.
By definition, $f(n) = O(n^3)$.

We set c=2 and $n_0$=1.
Then, $f(n)=2n^3+50n^2+50n\geq 2n^3=cn^3$ for $n\geq 1=n_0$.
By definition, $f(n) = \Omega(n^3)$.

**(c) (3 points) (Limit theorem & L'Hopital's rule)**

Given that that $h(n) = \ln n^3$, use the limit theorem to prove that $h(n) = o(n)$.

[*Hint:* Note that $\frac{d}{dx}(\ln x) = \frac{1}{x}$.]

Soln: $\lim_{n\to\infty} \frac{h(n)}{n} = \lim_{n\to\infty} \frac{\ln n^3}{n} = \lim_{n\to\infty} \frac{\frac{\mathrm{d}\, 3 \ln n}{\mathrm{dn}}}{\frac{\mathrm{dn}}{\mathrm{dn}}} = \lim_{n\to\infty} \frac{\frac{3}{n}}{1} = \lim_{n\to\infty} \frac{3}{n} = 0.$

Hence, $h(n) = o(n)$.

## Q2. (15 points)

**(a)** **(5 points)** Prove or disprove the following statement.
If f(n) = O(g(n)), then $a^{f(n)} = O(a^{g(n)})$ for any constant a>1. (You can assume both f(n)>0 and g(n)>0 for n>0.)
**[Show your working.]**

Answer: This is not correct. For example, f(n) = 2 lg n, g(n)= lg n and a=2. Then $a^{f(n)} = n^2$ and $a^{g(n)} = n$. Note that $n^2 \neq O(n)$.

## Q2. (continued)

**(b)** **(10 points)** Solve the following recurrences. Whenever possible, give your answers in Θ–notation. For simplicity, assume that $R(n)$, $S(n)$, $U(n)$, $V(n)$ are some small constants when $n \leq 4$.
**[Show your working.]**

**(i) (3 points)** $R(n) = 3R(n/6) + 5n^{0.5}$

Soln: $a=3$, $b=6$. So, $n^{\log_b a} = n^{\log_6 3} = n^{0.613}$.
Since $5n^{0.5} = O(n^{0.613})$, this is case 1. Hence, $R(n) = O(n^{0.615})$.

**(ii) (3 points)** $S(n) = 2S(n/2) - S(n/4) + 5n$
*(hint: $T(n)=S(n)-S(n/2)$.)*

Soln: By rearranging the formula, we have $S(n) - S(n/2) = S(n/2) - S(n/4) + 5n$.
By substituting $T(n)=S(n)-S(n/2)$, we have $T(n) = T(n/2) + 5n$.
By master theorem, we have $T(n)=\Theta(n)$.
Hence, $S(n)-S(n/2)=cn$ for some constant $c$.
By rearranging the equation, we have $S(n) =S(n/2)+cn$.
By master theorem, we have $S(n)=\Theta(n)$.

## Q2.(a) (continued)

**(iii) (4 points)** Computer *U(n)* where $U(n) = U(\frac{n}{4}) + V(\frac{3n}{4}) + n^2$ and $V(n) = 3V(\frac{n}{3}) + n$.

Soln: By master theorem, we have $V(n) = \Theta(n \log n)$. By substitute $V(\frac{3n}{4})$ by $cn \log n$, we have $U(n) = U(\frac{n}{4}) + cn \log n + n^2$.

By master theorem, we have $U(n) = \Theta(n^2)$.

## Q3. (10 points)

Consider a town with n men and n women seeking to get married to one another. Each man has a preference list that ranks all the women, and each woman has a preference list that ranks all the men.

The set of all 2n people is divided into 2 categories: good people and bad people. Suppose that for some number k, $1 \le k \le n-1$, there are k good men and k good women; thus there are n–k bad men and n–k bad women.

Everyone would rather marry any good person than any bad person. Formally, each preference list has the property that it ranks each good person of the opposite gender higher than each bad person of the opposite gender: its first k entries are the good people (of the opposite gender) in some order, and its next n–k entries are the bad people (of the opposite gender) in some order.

**(a)** **(4 points)** Suppose there exists a good man who is married to a bad woman in a stable matching S. Show that there exists a good woman who is married to a bad man.

Answer: By contrary, assume all k good women are married to k good men. Then, there is no good man who is married to a bad woman. We arrived at contradiction.

## Q3. (continued)

**(b)** **(6 points)** Show that in every stable matching, every good man is married to a good woman.

Answer: Assume by contradiction that there is a stable matching S, and in S, not all good man are married to a good woman.

So there exists a good man MG* who is married to a bad woman, say WB,. Meaning we have the pairing (MG* , WB) in S.

By (a), there exists a good woman, say WG* who is married to some bad man, saw MB. Meaning we have the pairing (MB , WG*) in S.

Then, clearly MG* and WG* forms an unstable pair in S since

MG* prefers WG* to WB, and

WG* prefers MG* to MB.

This leads to a contradiction. Hence, in S, every good men is married to a good women.

## Q4. (10 points)

**[Tri-Partitioning for a better Quicksort?]**
In *standard* Quicksort, we partition an array $A[1..n]$ about a pivot element $x$. Prof. Triple suggests that we might be able to do better if we use *two* pivot elements, instead of just one. (He thinks that a good *tripartite partition* works better, just like the *good tripartite industrial relations* in Singapore.)

**(a)** Suppose that you choose *two* elements $x$ and $y$ (for simplicity, assume that $x < y$) to partition the array $A[1..n]$ into three partitions: the left partition with all elements $\leq x$, the middle partition with all elements $> x$ and $< y$, and the right partition with all elements $\geq y$. Give an algorithm to perform this tri-partitioning with two pivots.
[*Hint:* Extend/Generalize the partitioning algorithm given in the lecture to do partitioning into three parts, small-elements, medium-elements, large-elements.]

Ans: A simple solution.

```
i ← PARTITION(A, p, q);
j ← PARTITION(A,i+1,q);
```

Then, p..i, i+1..j, j+1..q are the three parts.

## Q4. (continued)

**(b)** What is the best-case for Quicksort using tri-partitioning. In that case, give the corresponding recurrence and solve it to get the best-case running time.

Ans: In the best case, the tri-partitioning will give 3 equal parts. Then, the running time of the sorting algorithm is $T(n)=3T(n/3)+cn$.
By solving the recursion, we have $T(n) = O(n \log_3 n)$.

(*This page is blank. Use it for your answer, if needed.*)

**-- End of Paper --**